

Cambios de plataforma: acceso local y perfiles laborales

Guía técnica de implementación y archivos afectados

Autor: Manus AI

Fecha: 31 de agosto de 2026

Entorno objetivo: EasyPanel 2.33.2 sobre VPS de Google

Contenido

1	1. Resumen ejecutivo	1
2	2. Arquitectura de autenticación	1
	2.1 2.1 Alternativa elegida	1
	2.2 2.2 Flujo de ingreso	1
	2.3 2.3 Roles iniciales	1
3	3. Perfiles laborales	2
	3.1 3.1 Propósito	2
	3.2 3.2 Campos administrados	2
4	4. Cambios de backend y base de datos	2
	4.1 4.1 Archivos backend	2
	4.2 4.2 Tablas y columnas	3
5	5. Cambios de frontend	3
6	6. Instalación en PostgreSQL de EasyPanel	3
7	7. Variables y librerías	4
8	8. Pruebas y límites de validación	4
9	9. Seguridad y operación	4
10	10. Checklist de entrega	5
11	11. Actualización final de implementación	5
12	12. Correcciones posteriores y alcance operativo	5
13	13. Referencias técnicas	6

1 1. Resumen ejecutivo

Se incorporó una primera capa de acceso local para proteger la operación interna de Talento AISA. Las rutas administrativas —plazas, formularios, candidatos, informes, configuración, perfiles laborales y usuarios— continúan pasando por el layout protegido y ahora pueden utilizar una sesión local firmada mediante cookie HttpOnly. Los formularios públicos por identificador seguro permanecen accesibles sin sesión porque son enlaces destinados a candidatos.

El usuario inicial de testing se entrega mediante un seed SQL separado. La cuenta es `adminit@aisa.com.gt` y la contraseña temporal es `ADMIN`. La cuenta se marca con `password_change_required = true`; después del primer ingreso debe cambiarse desde Mi cuenta. Esta contraseña solo es válida para pruebas y debe sustituirse antes de cualquier uso operativo.

También se agregó el modelo reutilizable de perfiles laborales. Un perfil puede contener responsabilidades, requisitos, habilidades técnicas y blandas, conocimientos, nivel académico, rangos de experiencia, idiomas, licencias, disponibilidad, ubicación, rango salarial, modalidad de trabajo y criterios para razonamiento de IA.

2 2. Arquitectura de autenticación

2.1 2.1 Alternativa elegida

Se mantiene Manus OAuth como mecanismo compatible existente y se agrega autenticación local como alternativa operativa. El contexto del servidor intenta resolver primero la sesión local; si no existe, conserva el comportamiento OAuth anterior. Esto permite migrar sin invalidar integraciones existentes.

La sesión local se firma con `JWT_SECRET`, se guarda en la cookie `talento-claro-session`, expira en doce horas y se elimina al cerrar sesión. El hash de contraseña utiliza `scrypt` de Node.js con salt aleatorio. La contraseña no se almacena en texto plano.

2.2 2.2 Flujo de ingreso

Usuario → `/login` → `auth.localLogin` → PostgreSQL → `scrypt verify` → cookie HttpOnly → `/admin`

代码 1 Secuencia de autenticación local

El frontend ejecuta `trpc.auth.localLogin`. El backend busca una cuenta activa por correo, verifica el hash, firma un JWT local y establece la cookie. En cada solicitud posterior, `server/_core/context.ts` lee la cookie, consulta al usuario por ID y acepta la sesión solo si la cuenta está activa.

Si la cuenta tiene `password_change_required`, el login redirige a `/admin/account`. La recuperación utiliza un token hash con vencimiento de treinta minutos y un webhook opcional definido por `PASSWORD_RESET_WEBHOOK_URL`.

2.3 2.3 Roles iniciales

Rol	Alcance
Administrador	Acceso total, usuarios, perfiles, plazas, formularios, candidatos, informes y configuración.

Reclutador	Operación de plazas, candidatos, informes y consulta de perfiles; sin mantenimiento de usuarios ni configuración restringida.
Usuario genérico	Se conserva por compatibilidad, pero no puede usar procedimientos operativos protegidos.

Las guardas `adminProcedure` y `recruiterProcedure` se aplican en el backend. Ocultar un enlace no sustituye la autorización del servidor.

3 3. Perfiles laborales

3.1 3.1 Propósito

El módulo permite definir el perfil requerido antes de construir o ajustar el formulario de una plaza. La información del perfil es reutilizable y prepara la futura alimentación automática de reglas, preguntas y evaluación IA.

```

job_profiles ⇔ job_profile_positions ⇔ job_positions
job_profiles → form_questions → applications → evaluations

```

代码 2 Relación conceptual del perfil con la evaluación

3.2 3.2 Campos administrados

Grupo	Contenido
Identificación	Nombre, resumen y objetivo del puesto.
Funciones	Responsabilidades y requisitos obligatorios.
Competencias	Habilidades técnicas, blandas y conocimientos.
Formación	Nivel académico, idiomas, licencias y certificaciones.
Experiencia	Años mínimos y máximos, con soporte para criterios IA.
Condiciones	Disponibilidad, ubicación, rango salarial y modalidad.
Evaluación	Criterios de razonamiento, evidencia e incongruencias.

La primera capa de información del candidato seguirá proviniendo de las respuestas del formulario. Las conversaciones de WhatsApp y los documentos futuros, como el currículum vitae, podrán complementar y contrastar el perfil posteriormente.

4 4. Cambios de backend y base de datos

4.1 4.1 Archivos backend

Archivo	Cambio
<code>server/localAuth.ts</code>	Hash scrypt, verificación, emisión y lectura de JWT local, cookies y tokens de recuperación.
<code>server/db.ts</code>	Consulta <code>getUserById</code> para resolver sesiones locales.
<code>server/_core/context.ts</code>	Prioridad de sesión local y validación del usuario activo antes de OAuth.

server/routers.ts	Login, logout, cambio/recuperación de contraseña, usuarios y CRUD de perfiles.
scripts/create_admin_seed.mjs	Generador reproducible del hash y del seed inicial.
database/002_local_admin.sql	Seed idempotente del administrador inicial con hash scrypt.

4.2 4.2 Tablas y columnas

En `users` se agregaron `password_hash`, `password_change_required`, `password_updated_at`, `reset_token_hash`, `reset_token_expires_at` y `active`. Se añadieron `job_profiles` y `job_profile_positions`; esta última posee un índice único para impedir asociaciones duplicadas entre un perfil y una plaza.

La migración se encuentra en `drizzle/migrations/0003_lethal_lila_cheney.sql`. El seed del administrador se encuentra en `database/002_local_admin.sql`.

5 5. Cambios de frontend

Archivo	Cambio
client/src/pages/Login.tsx	Pantalla responsive de login local y solicitud de recuperación.
client/src/pages/Account.tsx	Cambio de contraseña, especialmente para la cuenta temporal.
client/src/pages/Users.tsx	Listado, creación, activación y desactivación de usuarios.
client/src/pages/Profiles.tsx	Alta, edición, búsqueda y definición de perfiles laborales.
client/src/App.tsx	Rutas <code>/login</code> , <code>/admin/account</code> , <code>/admin/users</code> y <code>/admin/profiles</code> .
client/src/components/DashboardLayout.tsx	Redirección al login local y navegación acorde al rol.

El formulario público `/apply/:token` se mantiene sin autenticación. El resto de la información administrativa requiere sesión y autorización backend.

6 6. Instalación en PostgreSQL de EasyPanel

La migración fue generada para PostgreSQL. Debe ejecutarse sobre la instancia PostgreSQL desplegada en EasyPanel, no sobre el TiDB utilizado por el preview interno. El orden operativo es:

1. Configurar `DATABASE_URL` en el servicio web de EasyPanel.
2. Redeployar la aplicación.
3. Ejecutar `drizzle/migrations/0003_lethal_lila_cheney.sql` sobre PostgreSQL.
4. Ejecutar `database/002_local_admin.sql` una sola vez.
5. Reiniciar o redeployar el servicio web.
6. Abrir `/login` y probar `adminit@aiaa.com.gt` con ADMIN.
7. Cambiar inmediatamente la contraseña desde `/admin/account`.

```
SELECT version();
SELECT email, role, active, password_change_required
```

```
FROM users
WHERE lower(email) = lower('adminit@aisa.com.gt');
```

El primer resultado debe identificar PostgreSQL. Si devuelve TiDB/MySQL, no debe ejecutarse la migración PostgreSQL hasta corregir la conexión del servicio.

7 7. Variables y librerías

Elemento	Ubicación	Uso
DATABASE_URL	EasyPanel/app	Conexión PostgreSQL.
JWT_SECRET	EasyPanel/app	Firma de sesión local.
PASSWORD_RESET_WEBHOOK_URL	EasyPanel/app	Entrega opcional de recuperación por correo.
pg + drizzle-orm/node-postgres	Backend	Pool y consultas PostgreSQL.
jose	Backend	JWT de sesión local.
Node crypto	Backend	scrypt, salt y tokens.
@trpc/react-query	Frontend	Mutaciones y consultas tipadas.

Las credenciales no se escriben en TypeScript ni en el repositorio. La conexión del servicio web debe apuntar al PostgreSQL de EasyPanel. Las credenciales nativas de PostgreSQL y OpenAI/ChatGPT de n8n se asignan desde la interfaz de n8n después de importar los workflows.

8 8. Pruebas y límites de validación

La comprobación TypeScript y la suite existente finalizaron correctamente después de los cambios: cuatro archivos de prueba y doce pruebas pasaron. El build de producción también finalizó correctamente.

La prueba real de la cuenta inicial requiere ejecutar la migración y el seed en la PostgreSQL de EasyPanel. No puede completarse contra el TiDB de preview porque sus tipos y sintaxis son incompatibles con el esquema PostgreSQL.

Debe verificarse manualmente que un usuario no autenticado no pueda consultar /admin, /admin/users, /admin/profiles, candidatos ni configuración; que un reclutador reciba FORBIDDEN al invocar procedimientos administrativos; y que el administrador pueda crear un usuario y un perfil.

9 9. Seguridad y operación

La contraseña ADMIN solo debe existir durante testing. El seed fuerza el cambio posterior y las contraseñas se almacenan con scrypt. Debe configurarse un JWT_SECRET propio, largo y aleatorio; el valor de desarrollo no es aceptable en producción.

La recuperación de contraseña requiere conectar PASSWORD_RESET_WEBHOOK_URL a un servicio de correo o workflow interno. Mientras esa variable no exista, la respuesta será genérica, pero no habrá canal para entregar el token al usuario.

Antes de producción se recomienda añadir límite de intentos de login, sesiones revocables, control de dispositivos, SMTP formal, permisos por área y auditoría específica para cambios de perfiles.

10 10. Checklist de entrega

☐	Configurar DATABASE_URL de PostgreSQL en EasyPanel.
☐	Ejecutar migración 0003 y verificar <code>SELECT version()</code> .
☐	Ejecutar seed del administrador inicial.
☐	Ingresar con <code>adminit@aisa.com.gt</code> / ADMIN.
☐	Cambiar la contraseña temporal.
☐	Crear usuarios reclutadores y validar permisos.
☐	Crear el primer perfil laboral y asociarlo a una plaza.
☐	Configurar el webhook de recuperación de contraseña.

11 11. Actualización final de implementación

Después de la revisión de calidad se completaron las siguientes correcciones adicionales:

Archivo	Actualización
<code>client/src/pages/ResetPassword.tsx</code>	Formulario para consumir el token de recuperación y establecer una nueva contraseña.
<code>client/src/pages/App.tsx</code>	Ruta pública <code>/reset-password</code> .
<code>client/src/pages/Users.tsx</code>	Edición de usuarios existentes y desactivación desde la misma vista.
<code>client/src/pages/Profiles.tsx</code>	Edición persistente mediante ID, baja lógica y campos separados de disponibilidad, ubicación, modalidad y salario.
<code>server/routers.ts</code>	Consulta <code>profiles.forPosition</code> para conectar el perfil laboral con el constructor del formulario.
<code>server/localAuth.test.ts</code>	Pruebas de hash scrypt, verificación de contraseñas y tokens de recuperación.

La verificación posterior a estas modificaciones concluyó con `pnpm check` sin errores, 14 pruebas Vitest exitosas y build de producción exitoso. El único paso que continúa pendiente es ejecutar la migración y el seed en la PostgreSQL real de EasyPanel; el preview interno sigue apuntando a TiDB y no debe utilizarse para aplicar la migración PostgreSQL.

12 12. Correcciones posteriores y alcance operativo

Se agregó `notifyPasswordReset` en `server/localAuth.ts`, que envía el token a `PASSWORD_RESET_WEBHOOK_URL`, devuelve estado de entrega y evita que un fallo externo exponga el token o interrumpa la respuesta genérica. La cobertura de `server/localAuth.test.ts` simula una respuesta HTTP 202 y valida URL y payload.

En `Users.tsx` se añadieron búsqueda por nombre/correo, filtros de rol y estado, edición de usuarios existentes y preservación del estado durante la actualización. En `Profiles.tsx` se añadieron filtros de estado, indicador activo/inactivo, reactivación y persistencia explícita de disponibilidad, ubicación, rango salarial y modalidad.

El workflow `n8n-workflows/02_agente_plaza_template.json` ahora incorpora el perfil laboral activo asociado a la plaza en `evaluation_input.profile`. El prompt del nodo Evaluar respuestas abiertas recibe ese contexto junto con respuestas y reglas, y la validación estructural de los cuatro workflows continúa pasando.

La migración y el seed del administrador siguen pendientes de ejecución en la PostgreSQL real de EasyPanel; el preview interno no debe utilizarse para ese paso mientras continúe apuntando a TiDB.

13 13. Referencias técnicas

- [1] [EasyPanel App Service](<https://easypanel.io/docs/services/app>). Configuración de servicios, variables de entorno y despliegue.
- [2] [n8n: Use environment variables](<https://docs.n8n.io/deploy/host-n8n/configure-n8n/basic-configuration/use-environment-variables/>). Variables de entorno en instalaciones self-hosted.
- [3] [PostgreSQL Documentation](<https://www.postgresql.org/docs/current/>). Tipos, transacciones y conexiones.